

Chapter 14: SQL Aggregation & Grouping

```
SELECT department,  
       COUNT(*)FROM EmployeeWHERE COUNT(*) > 2GROUP BY department;
```

Error

Why?

Because at the time SQL executes WHERE
Groups DO NOT EXIST YET
COUNT(*) hasn't been calculated.

FROM



WHERE



GROUP BY



Aggregate Functions



HAVING



SELECT



ORDER BY



LIMIT

Notice

HAVING comes after aggregation.

14.6.4 Syntax

```
SELECT group_column,  
       aggregate_function(column)FROM tableGROUP BY group_columnHAVING  
aggregate_condition;
```

Departments with more than 2 employees.

```
SELECT department,  
       COUNT(*) AS total_employeesFROM EmployeeGROUP BY departmentHAVING  
COUNT(*) > 2;
```

WHERE + GROUP BY + HAVING

This is the pattern used in a huge number of interview questions.

Suppose

Employee

Name	Department	Salary
Arpit	ECE	70000
Aadz	ECE	75000
Rahul	ECE	30000
Riya	CSE	60000
Priya	CSE	65000
Aman	ME	40000

Question

Consider only employees earning more than ₹50,000.
Among them, show departments having at least two employees.

Query

```
SELECT department,  
       COUNT(*) AS total_employees  
FROM Employee  
WHERE salary > 50000  
GROUP  
BY department  
HAVING COUNT(*) >= 2;
```

What DISTINCT Actually Does

DISTINCT simply removes duplicate rows.

Think of it as



No groups are created.
No aggregation happens.

Example

Table student

student	branch
Arpit	ECE
Aadz	ECE
Rahul	CSE

Query

```
SELECT DISTINCT branchFROM Students;
```

SQL internally sees

ECE

ECE

CSE

Removes duplicates

ECE

CSE

Finished.

14.8.3 What GROUP BY Actually Does

GROUP BY creates groups.

Example

```
SELECT branchFROM StudentsGROUP BY branch;
```

Internal grouping

ECE

Arpit

Aadz

ECE

Riya

Merged into one ECE group

CSE

Rahul

Priya

Now SQL has groups, not just unique values.

GROUP BY Supports Aggregates

```
SELECT department, COUNT(*),  
SUM(salary),  
AVG(salary),  
MAX(salary),  
MIN(salary) FROM Employee GROUP BY department;
```

DISTINCT cannot do this.

Can DISTINCT replace GROUP BY?

Only when

- no aggregates exist
- only unique rows are required

Why Do We Need Nested Grouping?

Suppose Employee table

Employee	Department	Salary
Arpit	ECE	70000
Aadz	ECE	75000
Rahul	ECE	68000
Riya	CSE	60000
Priya	CSE	65000
Aman	ME	45000

Question

Find the highest department average salary.

Notice carefully.

We are NOT asking

Highest employee salary.

We're asking

Highest department average.

That requires

Step 1: Calculate department averages.



Step 2: Find the maximum average.

This is an aggregate over another aggregate.

```
SELECT MAX(max_salary)FROM
(
  SELECT department,
         MAX(salary) AS max_salary
  FROM Employee
  GROUP BY department
) t;
```

Department with Maximum Employees

Interview favorite.

Employee table

Department

ECE
ECE
ECE
ECE
CSE
CSE
ME

Step 1

Count employees.

```
SELECT department,COUNT(*) AS cntFROM EmployeeGROUP BY department;
```

Output

Department	Count
ECE	4
CSE	2
ME	1

Now

Find largest count.

```
SELECT MAX(cnt)FROM
(
  SELECT department,
    COUNT(*) AS cnt
  FROM Employee
  GROUP BY department
) t;
```

Suppose question says

Which department has the most employees?

Now we cannot simply use MAX.

Instead

```
SELECT department,COUNT(*) AS cntFROM EmployeeGROUP BY departmentORDER
BY cnt DESCLIMIT 1;
```

Common Mistakes

✗ Nested aggregate in same query block

```
SELECT MAX(AVG(salary))FROM EmployeeGROUP BY department;
```

✓ Correct

```
SELECT MAX(avg_salary)FROM
(SELECT department,
AVG(salary) avg_salaryFROM EmployeeGROUP BY department
)t;
```

✗ Forgetting alias for derived table

```
SELECT MAX(cnt)FROM
(SELECT department,COUNT(*) cntFROM EmployeeGROUP BY department
);
```

Most databases require an alias.

✓ Correct

```
SELECT MAX(cnt)FROM
(SELECT department,COUNT(*) cntFROM EmployeeGROUP BY department
) t;
```